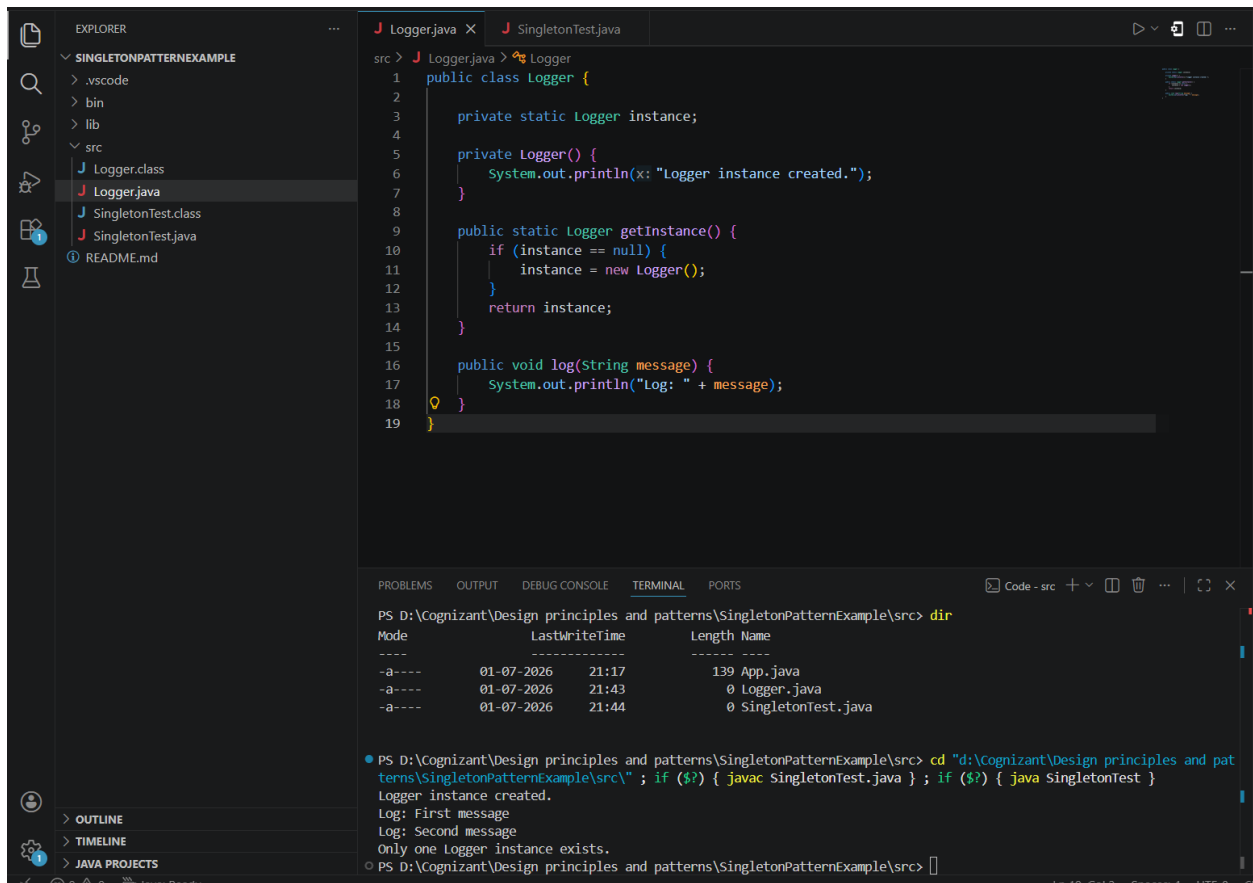


Exercise 1: Implementing the Singleton Pattern



The screenshot displays an IDE with the following components:

- EXPLORER:** Shows the project structure for `SINGLETONPATTERNEXAMPLE`. The `src` directory contains `Logger.class`, `Logger.java`, `SingletonTest.class`, `SingletonTest.java`, and `README.md`.
- Editor:** Displays `Logger.java` with the following code:

```
1 public class Logger {
2
3     private static Logger instance;
4
5     private Logger() {
6         System.out.println(x: "logger instance created.");
7     }
8
9     public static Logger getInstance() {
10         if (instance == null) {
11             instance = new Logger();
12         }
13         return instance;
14     }
15
16     public void log(String message) {
17         System.out.println("Log: " + message);
18     }
19 }
```
- TERMINAL:** Shows the execution of the Singleton pattern. The command `cd "d:\Cognizant\Design principles and patterns\SingletonPatternExample\src" & dir` is executed, resulting in the following output:

Mode	LastWriteTime	Length	Name
-a----	01-07-2026 21:17	139	App.java
-a----	01-07-2026 21:43	0	Logger.java
-a----	01-07-2026 21:44	0	SingletonTest.java

Below the directory listing, the output of the `java SingletonTest` command is shown:

```
Log: First message
Log: Second message
Only one Logger instance exists.
```

VS Code interface showing a Java project named SingletonPatternExample. The Explorer sidebar on the left shows the project structure: .vscode, bin, lib, src, and README.md. The src folder contains Logger.class, Logger.java, SingletonTest.class, and SingletonTest.java. The main editor displays SingletonTest.java with the following code:

```
src > J SingletonTest.java > SingletonTest
1 public class SingletonTest {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Logger logger1 = Logger.getInstance();
7         logger1.log(message: "First message");
8
9         Logger logger2 = Logger.getInstance();
10        logger2.log(message: "Second message");
11
12        if (logger1 == logger2) {
13            System.out.println(x: "Only one Logger instance exists.");
14        } else {
15            System.out.println(x: "Different Logger instances exist.");
16        }
17    }
```

The bottom panel shows the TERMINAL output:

```
PS D:\Cognizant\Design principles and patterns\SingletonPatternExample\src> dir
Mode                LastWriteTime         Length Name
-----
-a----          01-07-2026   21:17             139 App.java
-a----          01-07-2026   21:43              0 Logger.java
-a----          01-07-2026   21:44              0 SingletonTest.java

• PS D:\Cognizant\Design principles and patterns\SingletonPatternExample\src> cd "d:\Cognizant\Design principles and pat
terns\SingletonPatternExample\src" ; if ($?) { javac SingletonTest.java } ; if ($?) { java SingletonTest }
Log: First message
Log: Second message
Only one Logger instance exists.
PS D:\Cognizant\Design principles and patterns\SingletonPatternExample\src>
```

The status bar at the bottom indicates: Ln 17, Col 2, Spaces: 4, UTF-8, C.

Exercise 2: Implementing the Factory Method Pattern

Document.java

```
public interface Document {  
    void open();  
}
```

DocumentFactory.java

```
public interface Document {  
    void open();  
}
```

ExcelDocument.java

```
public class ExcelDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Excel Document Created");  
    }  
}
```

ExcelDocumentFactory.java

```
public class ExcelDocumentFactory extends DocumentFactory {
```

```
@Override  
public Document createDocument() {  
    return new ExcelDocument();  
}  
}
```

FactoryTest.java

```
public class FactoryTest {  
  
    public static void main(String[] args) {  
  
        DocumentFactory wordFactory = new WordDocumentFactory();  
        Document word = wordFactory.createDocument();  
        word.open();  
  
        DocumentFactory pdfFactory = new PdfDocumentFactory();  
        Document pdf = pdfFactory.createDocument();  
        pdf.open();  
  
        DocumentFactory excelFactory = new ExcelDocumentFactory();  
        Document excel = excelFactory.createDocument();  
    }  
}
```

```
        excel.open();  
    }  
}
```

PdfDocument.java

```
public class PdfDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("PDF Document Created");  
    }  
}
```

PdfDocumentFactory.java

```
public class PdfDocumentFactory extends DocumentFactory {  
  
    @Override  
    public Document createDocument() {  
        return new PdfDocument();  
    }  
}
```

WordDocument.java

```
public class WordDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Word Document Created");  
    }  
}
```

WordDocumentFactory.java

```
public class WordDocumentFactory extends DocumentFactory {  
  
    @Override  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

